

Generative AI in Practice: Hands-on Development of RAG Systems using LangChain

Waqar Ashiq

Lecturer, SE Department

University of Management and Technology, Lahore

Outline of the Lecture

- Introduction to Generative AI and LLMs
- RAG Architecture Overview
- Introduction to LangChain Framework
- Vector Databases: FAISS, Pinecone, ChromaDB
- Building RAG Pipelines with LangChain
- Prompt Engineering and Templates
- Deploying the Application
- Q&A / Discussion

Introduction to Generative AI and LLMs

Generative

create **new** content
(audio, code, images, **text**, video)

Artificial Intelligence

automatically
using a computer program

GENERATIVE AI TOOLS

ChatGPT

- Nov. 2022
- Developed by OpenAI
- <https://chat.openai.com/>

Bing AI

- Feb. 2023
- Developed by Microsoft
- <https://www.bing.com>

Google Bard

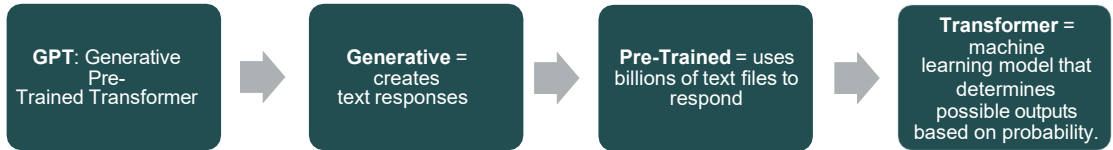
- Mar. 2023
- Developed by Google
- <https://bard.google.com/>

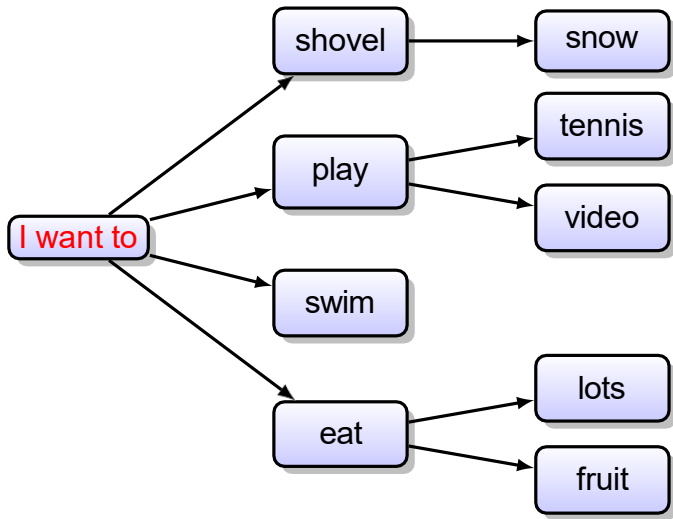
Others

- Notion, Jasper, DALL-E, DeepSeek and many more.

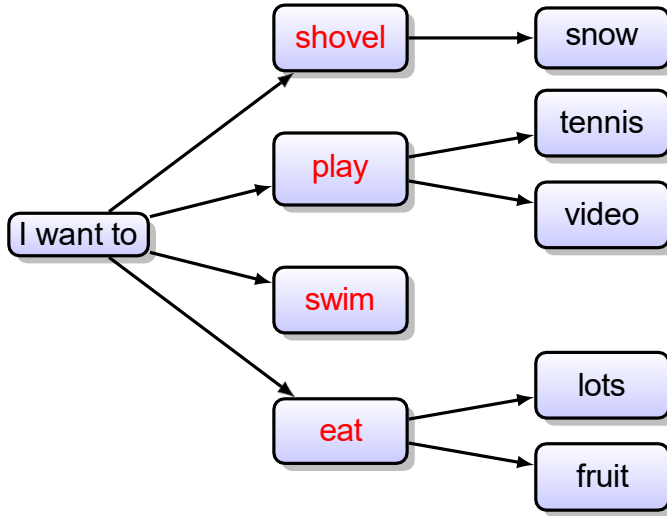
HOW DOES GENERATIVE AI WORK?

- **Generative AI (Gen AI):** Refers to deep-learning models that can generate high-quality text, images, and other content based on the data they are trained on.
- **ChatGPT: Large Language Model-based chatbot**

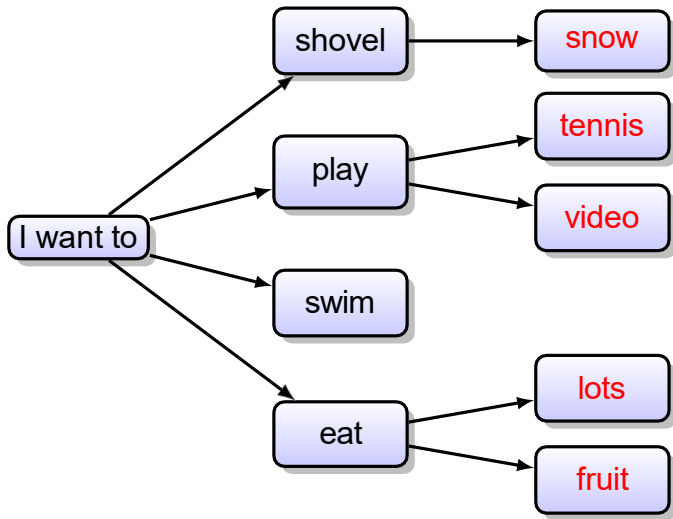




Given sequence of words so far (**context**), predict what comes **next**.

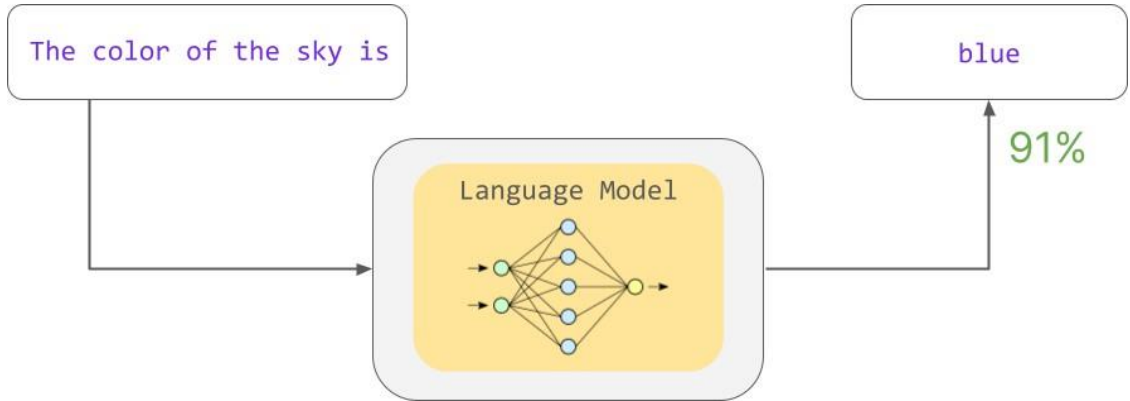


Given sequence of words so far (**context**), predict what comes **next**.



Given sequence of words so far (**context**), predict what comes **next**.

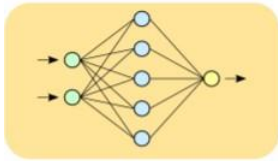
Language Modeling



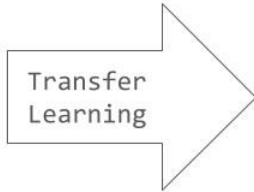
Given sequence of words so far (**context**), predict what comes **next**.

Exploiting a Pre-trained Model: Fine-Tuning

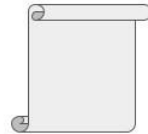
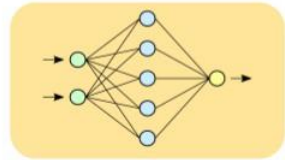
Pre-trained Model



Generic data



Fine-Tuned Model



Domain or task
specific data

How **good** can a language model become?

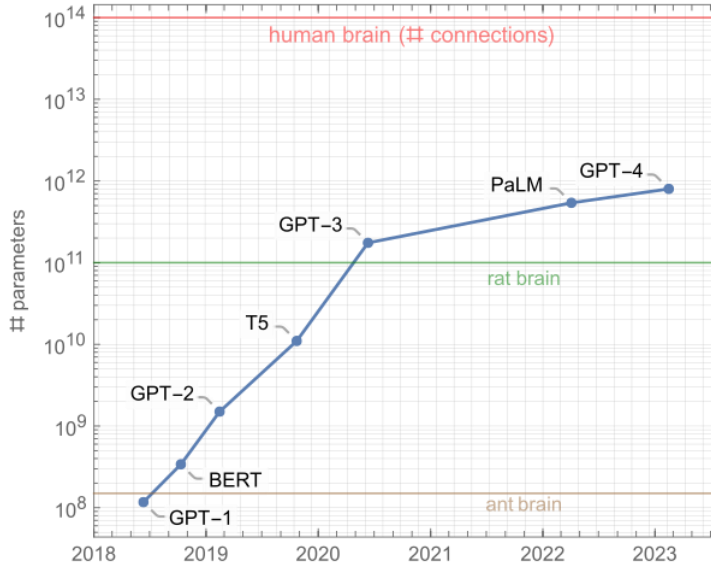
The bigger the better!

How **good** can a language model become?

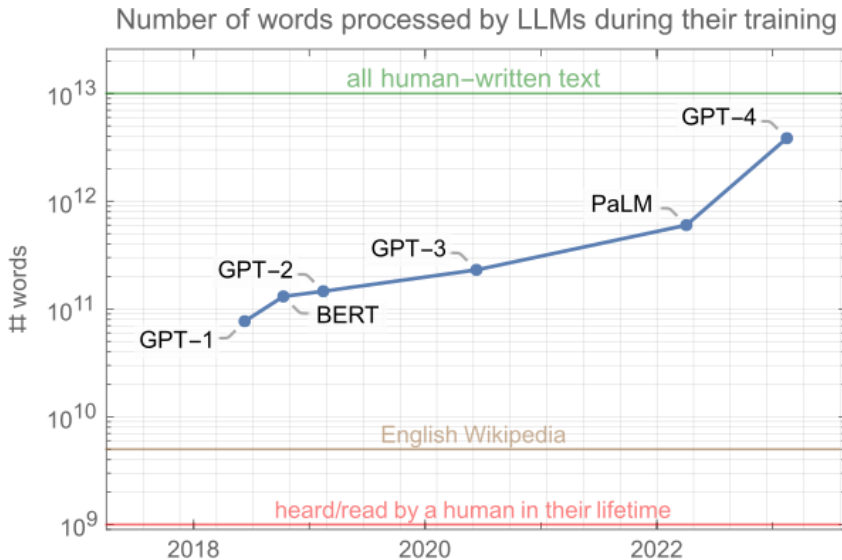
The bigger the better!

Since 2018, we have witnessed an extreme increase in model sizes.

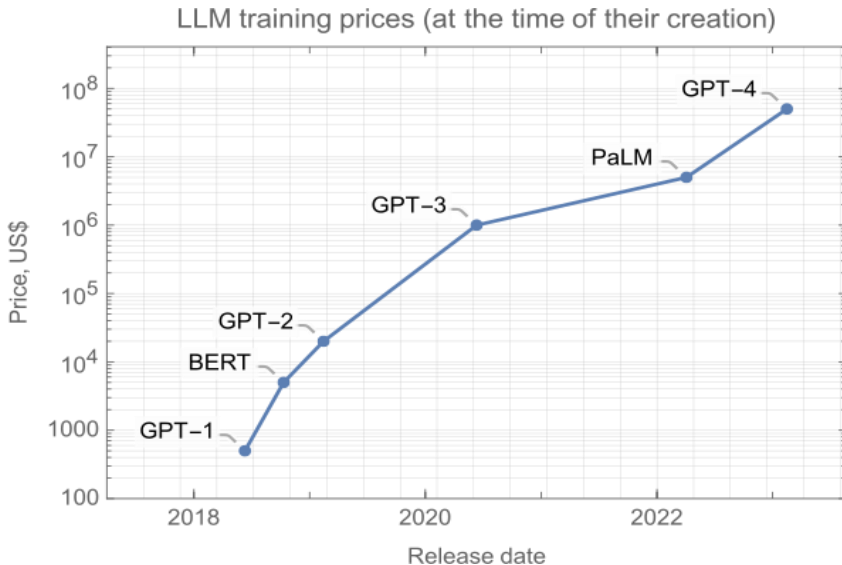
From Language Models to Large Language Models



From Language Models to Large Language Models



From Language Models to Large Language Models



Retrieval-Augmented Generation (RAG) Architecture Overview

When answering questions or generating text, it first **retrieves relevant information** from a large number of documents, and then LLMs generates answers based on this information.

By attaching a **external knowledge base**, there is no need to retrain the entire large model for each specific task.

The RAG model is especially suitable for **knowledge-intensive** tasks.

Domain-specific accurate answering

Frequent updates of data

Traceability and explainability of generated content

Controllable Cost

Privacy protection of data

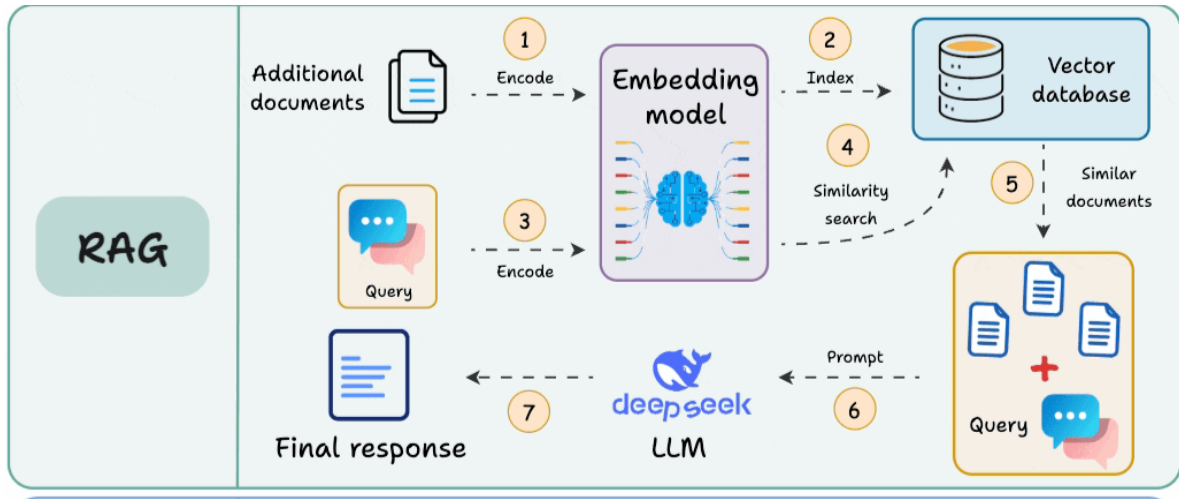
Drawbacks of LLM

- Hallucination
- Outdated information
- Low efficiency in parameterizing knowledge
- Lack of in-depth knowledge in specialized domains
- Weak inferential capabilities

Components of a RAG System

- Document Loader
- Embedding Model
- Vector Store
- Retriever
- Language Model
- Prompt Template
- Output Parser

RAG Architecture Overview



Introduction to LangChain Framework

Overview and Capabilities

- **LangChain** is an open-source framework designed to simplify the development of applications powered by Large Language Models (LLMs).
- It enables the chaining of components such as **prompts, LLMs, retrievers, tools, and memory** to build complex workflows.
- Supports retrieval-based, conversational, and agentic applications.

Modularity and Integrations

- Highly modular: you can plug in your own models, data sources, and tools.
- **Integrates seamlessly with:**
 - **LLM providers:** OpenAI, Hugging Face, etc.
 - **Embeddings:** OpenAI, Hugging Face, Sentence Transformers
 - **Vector databases:** FAISS, Pinecone, Chroma, etc.
 - **Document loaders:** PDFs, CSVs, websites, Notion, Google Docs, etc.
 - **Frameworks:** Streamlit, Gradio, FastAPI

Common LangChain Components

- **Chains:** Combine multiple steps (e.g., input → retrieval → prompt → LLM → output)
- **Tools:** External APIs or functions the LLM can call (e.g., calculator, web search)
- **Agents:** LLMs that can reason and decide which tools or chains to use dynamically
- **Retrievers:** Fetch relevant documents or chunks from a vector store
- **Prompt Templates:** Structured prompts with dynamic variables
- **Memory:** Store conversation history or context for stateful interactions

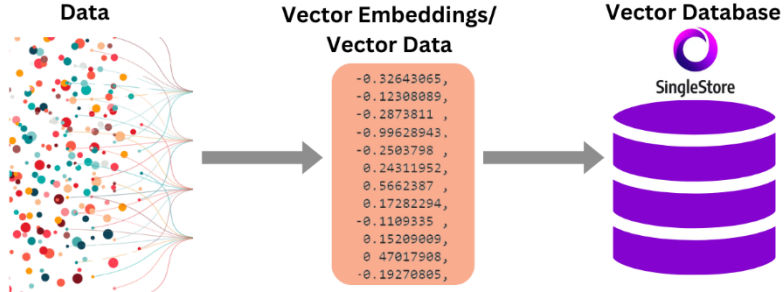
Vector Databases

Vector Databases

Vector databases are specialized databases designed to **store and manage vectorized data**, such as **embeddings** generated from text. They enable efficient similarity search, which is crucial in applications like RAG, where relevant documents need to be quickly retrieved based on their semantic similarity to a query.

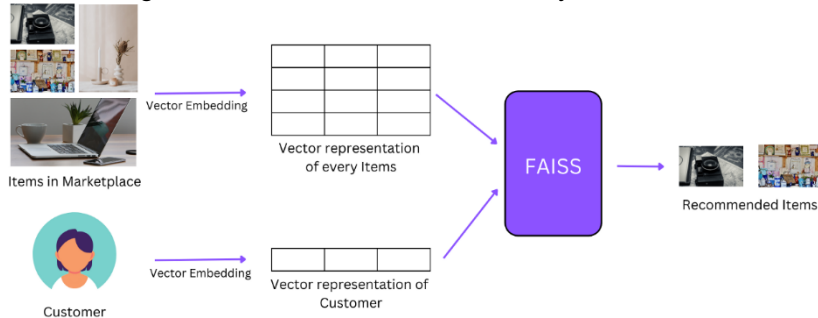
In the context of LangChain, vector databases are used to store the embeddings of document chunks, allowing for fast and accurate retrieval during the generation process. By storing vectors in a dedicated database, we can scale the retrieval process to handle large datasets while maintaining high performance.

Vector Databases



FAISS Databases

FAISS (Facebook AI Similarity Search) is an open-source library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors. FAISS is widely used in machine learning applications that involve searching for similar items in large datasets, such as image retrieval, recommendation systems, and natural language processing.



Building RAG Pipelines with LangChain

Building RAG Pipelines with LangChain

1. Load and Prepare Documents

- Use LangChain's DocumentLoader to load data from sources like PDFs, web pages, or Notion.
- Split large documents into manageable chunks (e.g., using RecursiveCharacterTextSplitter).

2. Generate Embeddings

- Choose an embedding model (e.g., OpenAI, Hugging Face, or Sentence Transformers).
- Convert text chunks into dense vectors (embeddings).

3. Store Embeddings in a Vector Database

- Select a vector store: FAISS, Pinecone, Chroma, etc.
- Store the document vectors along with metadata (e.g., source, page number).

4. Set Up a Retriever

- Connect the vector store to a Retriever object.
- The retriever fetches the top-k most relevant chunks given a query.

5. Define a Prompt Template

- Create a reusable prompt with variables (e.g., context, question).

6. Connect to an LLM

- Use OpenAI, DeepSeek, or other supported LLMs.
- Combine retriever output with prompt and send it to the model.

7. Build the Chain

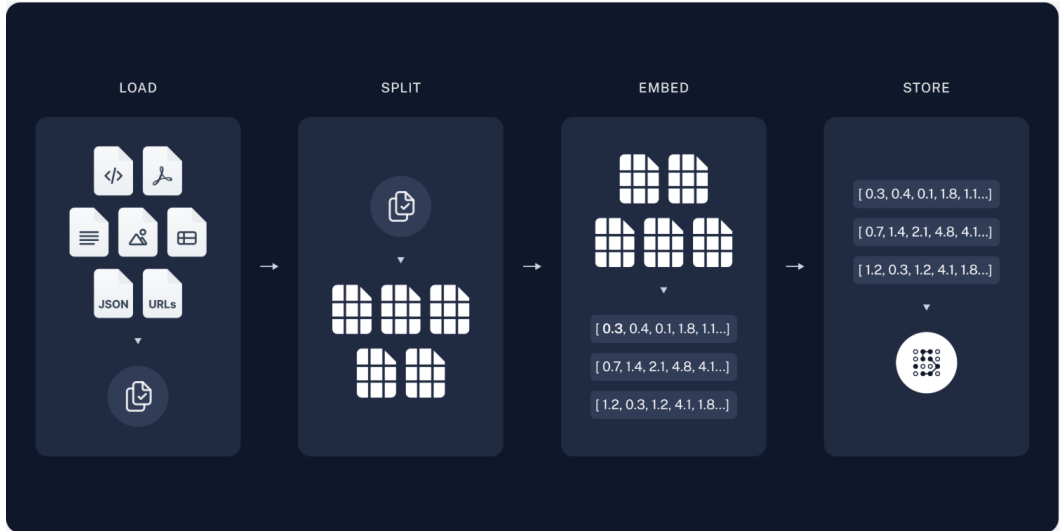
- Use LangChain's RetrievalQA or ConversationalRetrievalChain to orchestrate:
- Query → Retrieve → Prompt → Generate → Output

8. Deploy the RAG Application

- Wrap the chain in a web interface using Gradio, Streamlit, or FastAPI.
- Optional: Add memory, feedback loop, logging, or authentication.

How RAG Works in LangChain

In LangChain, RAG can be implemented using components like Loader, retrievers, text splitters, and vector stores. The process typically involves:

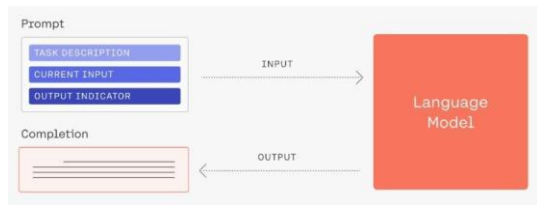
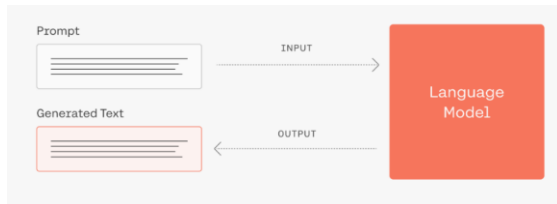


- Loading and splitting documents into manageable chunks.
- Embedding these chunks into vector representations using models like OpenAI embeddings.
- Storing these vectors in a vector database for efficient retrieval.
- Using a retrieval model to fetch relevant document chunks in response to a query.
- Generating text based on the retrieved chunks to provide accurate and contextually relevant answers.

Prompt Engineering and Templates

Prompt Engineering

- Prompt is a piece of text that you provide to a language model to get a specific response or output
- Prompt can be a sentence, a paragraph, or entire article
- Serves as starting point for the model to generate new content
- Constraints or focusses interaction



zero-shot, one-shot and few-shot learning

- **zero-shot learning:** prompting a language model with an instruction. No examples or demonstrations of the task you want it to perform
- Gives instruction in natural language
- **one-shot:** it includes one demonstration
- **few-shot (in-context)** learning includes many demonstrations 10+
- **Chain-of-Thought (CoT):** Prompting can be further improved by instructing the model to reason about the task when responding

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 cheese => prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer example
3 cheese => prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer examples
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => prompt
```

Thank You
Q&A / Discussion